Experiment 1

Aim

- Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.
- Write a program to implement Missionaries and Cannibals problem.
  Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

Theory

Water Jug Problem

The Water Jug problem is an AI problem that involves problem solving using state transitions and constraints. Here two jugs with capacities of 3 liters and 5 liters respectively are used to obtain exactly 4 liters of water in one of the jugs while other becomes empty. The jugs can be filled, emptied, or used to transfer water between each other. The program simulates all valid jug operations based on user input. At each step, the state of both jugs is updated according to the selected operation. The min() function ensures that no jug overflows during water transfer. The process continues until the target amount is achieved. This problem demonstrates logical reasoning, state representation, and constraint-based decision making. It helps in understanding how real-world problems can be solved computationally using simple rules and transitions.

Missionaries and Cannibals Problem

The Missionaries and Cannibals problem is another AI problem that demonstrates state space search and constraint satisfaction. Here three missionaries and three cannibals must cross a river using a boat that can carry a maximum of two people. The main constraint is that missionaries should never be outnumbered by cannibals on either side of the river, as it would lead to an unsafe situation.

The solution is implemented using Breadth First Search (BFS), which step by step explores all possible valid states. Each state is represented as a tuple containing the number of missionaries on the left bank, the number of cannibals on the left bank, and the position of the boat. A validity function ensures that unsafe or impossible states are discarded. BFS uses a queue to store states and a visited set to avoid revisiting previously explored states. Parent tracking is used to achieve the final solution path once the goal state is reached. BFS

guarantees the shortest and safest sequence of moves to successfully transport all missionaries and cannibals to the right bank.

Algorithm:

Water jug problem

Step 1: Start the program.

Step 2: Initialize values - Jug X capacity = 3 liters

Jug Y capacity = 5 liters

Water in both jugs = 0

Target = 4 liters

Step 3:

Display menu of possible operations:

1. Fill Jug X
2. Fill Jug Y
3. Empty Jug X
4. Empty Jug Y
5. Pour water from Jug X to Jug Y
6. Pour water from Jug Y to Jug X

Step 4:Execute the selected operation and update jug values. Repeat the process until the target amount (4 liters) is obtained in jug Y and jug X = 0

Step 5:If target is achieved, display success message and final values and stop.

Step 6:End program

Algorithm: Missionaries and Cannibals (BFS)

Step 1:Start the program.

Step 2: Define a validity function to check safe states where missionaries are not outnumbered by cannibals on either bank and if their numbers aren't less than 0 or greater than 3.

Step 3: Initialize the start state as (3, 3, 0) and the final state as (0, 0, 1). Boat on left side represents 0 and on right side represents 1.

Step 4: Create a queue and insert the start state. Mark the start state as visited.

Step 5:
Define all possible boat movements:
(1,0), (2,0), (0,1), (0,2), (1,1).

Step 6:
While the queue is not empty:

Remove the front state from the queue.

If it is the goal state, reconstruct and print the path using parent information and stop.

Step 7:
For each possible movement:

Generate a new state depending on the boat position.

Check if the new state is valid and not visited.

Mark it as visited, store its parent, and add it to the queue.

Step 8: End the program.

Dataset Description:

Source – given by user

2 jug problem

Target = desired amount of water

Capx = capacity of jug x

Capy = capacity of jug y

J1 = current water in jug 1

J2 = current water in jug 2

Transfer = Amount of water being transferred

Cannibals and missionaries

Missionaries = Number of missionaries on the left bank.

Cannibal = number of cannibals on left bank

Boat position = (0=left bank,1=right bank)

Start = initial state(3,3,0)

Finish = Final state (0,0,1)

Queue = store state values

Parent = stores parent state for final path reconstruction

Visited = visited set values

Movement = possible movement positions of boat with missionaries and cannibals

New_m = new number of missionaries on left bank after movement of few to the other bank.

New_c = new number of cannibals on left bank after movement of few to the other bank.

New_b = new position of boat

Implementation:

2 jug problem

```
target=4
capx=3
capy=5
j1=0
j2=0

while(True):
  if j1==0 and j2==target:
   print("Target achieved")
   break;
  step = int(input("Give step to execute " ))

  if step == 1:
```

```
 j1=capx
 print("fill x")
 print(f"x={j1} amd y={j2}")

elif step == 2:
  j2=capy
  print("fill y")
  print(f"x={j1} amd y={j2}")

elif step == 3:
  j1=0
  print("empty x")
  print(f"x={j1} amd y={j2}")

elif step == 4:
  j2=0
  print("empty y")
  print(f"x={j1} amd y={j2}")

elif step == 5:
  transfer = min(j1,capy-j2)
  j2+=transfer
  j1-=transfer
  print("x to y")
  print(f"x={j1} amd y={j2}")

elif step == 6:
  print("y to x")
  transfer = min(j2,capx-j1)
  j1+=transfer
  j2-=transfer
  print(f"x={j1} amd y={j2}")
```

```
  else:
    print("invalid choice")
    break;

print(f"x={j1} amd y={j2}")
```

Cannibals and missionaries:

```
from collections import deque

def validity(missionaries, cannibals):
    if missionaries < 0 or cannibals < 0 or missionaries > 3 or cannibals >
3:
        return False
    if missionaries > 0 and missionaries < cannibals:
        return False
    if (3 - missionaries) > 0 and (3 - missionaries) < (3 - cannibals):
        return False
    return True

def crossing():
    start = (3, 3, 0)
    finish = (0, 0, 1)

    queue = deque([start])
    visited = {start}
    parent = {}

    movement = [(1,0),(2,0),(0,1),(0,2),(1,1)]

    while queue:
        missionaries, cannibals, boat = queue.popleft()

        if (missionaries, cannibals, boat) == finish:
```

```
            path = []
            while (missionaries, cannibals, boat) in parent:
                path.append((missionaries, cannibals, boat))
                missionaries, cannibals, boat = parent[(missionaries,
cannibals, boat)]
            path.append(start)
            path.reverse()
            print("Final correct Path:")
            for p in path:
                print(p)
            return

        for moving_m, moving_c in movement:
            if boat == 0:
                 new_m, new_c, new_b = missionaries-moving_m, cannibals-
moving_c, 1
            else:
                new_m, new_c, new_b = missionaries+moving_m,
cannibals+moving_c, 0

            if validity(new_m, new_c) and (new_m, new_c, new_b) not in
visited:
                visited.add((new_m, new_c, new_b))
                parent[(new_m, new_c, new_b)] = (missionaries, cannibals,
boat)
                queue.append((new_m, new_c, new_b))

crossing()
```

Output:

```
Give step to execute 2
fill y
x=0 amd y=5
Give step to execute 6
y to x
x=3 amd y=2
Give step to execute 3
empty x
x=0 amd y=2
Give step to execute 6
y to x
x=2 amd y=0
Give step to execute 2
fill y
x=2 amd y=5
Give step to execute 6
y to x
x=3 amd y=4
Give step to execute 3
empty x
x=0 amd y=4
Target achieved
x=0 amd y=4
```

```
Final correct Path:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)
```

Result analysis:

The output of both programs successfully demonstrates correct problem solving using state transitions and constraints. In the Missionaries and Cannibals problem, BFS algorithm ensures that the shortest valid path is obtained. In the Water Jug problem, the output shows the step-by-step operations performed on the jugs until the target amount of water is achieved. Both outputs confirm that the constraints of the problems are properly handled and the desired goals are reached.

Conclusion:

From this experiment, we learned how classic AI problems can be solved using state space representation and rule based transitions while following constraints . The use of Breadth First Search helped in understanding optimal problem solving. We also learned how constraints are applied to avoid invalid states. The experiment improved our logical thinking and strengthened our understanding of search algorithms and problem modeling using Python.